

```
graph TD; pages --> components; pages --> composables; components --> composables; composables --> stores; stores --> utils; stores --> boot; stores --> services; subgraph boot; http_client[HTTP client<br/>(cookies, error translation)]; i18n[i18n<br/>(EN / DE catalogues)]; router_guard[router guard<br/>(authentication)]; end; subgraph services_api [services/api]; generated_api[Generated OpenAPI<br/>client types]; end; utils -.-> boot; boot -.-> generated_api;
```

The diagram illustrates the architecture of a Single-page application (Vue 3 + Quasar). It shows a hierarchical structure where **pages** and **components** feed into **composables** (which include `useContracts`, `usePermissions`, `useDeadlines`, and `useApprovals`). These composables then feed into **stores (Pinia)**. The stores are then used by **utils (pure functions)**, the **boot** module (which includes an **HTTP client** for cookies and error translation, **i18n** for EN / DE catalogues, and a **router guard** for authentication), and the **services/api** module (which generates OpenAPI client types). The **utils** module and the **boot** module both have dependencies on the **services/api** module.

```
utils
(pure functions)
```

stores (Pinia)

```
composables
(useContracts, usePermissions,
useDeadlines, useApprovals)
```

components

pages

```
graph TD; boot[boot] --> http_client["HTTP client  
(cookies, error translation)"]; boot --> i18n["i18n  
(EN / DE catalogues)"]; boot --> router_guard["router guard  
(authentication)"];
```

HTTP client
(cookies, error translation)

i18n
(EN / DE catalogues)

router guard
(authentication)

The diagram shows a box labeled **services/api** with an arrow pointing to a box labeled **Generated OpenAPI client types**. A dashed line also points from the **Generated OpenAPI client types** box back to the **services/api** box.

Generated OpenAPI
client types

```
/api (same origin,  
session cookies)
```

Backend REST API

types regenerated from
the OpenAPI document

Unit tests target stores, composables and utils; the .vue components stay thin.